NEAR HACKATHON | TRACK 07 AGENT ECONOMY + TRACK 06 AI SECURITY

Tollgate

Verify before you pay.

A payment firewall that sits between an AI agent and its wallet. It checks every machine payment request against a spending mandate and against independent facts before any money moves.

Product Requirements Document and System Architecture

Version	1.2 draft (replaces 1.1 of 26 Oct)
Date	26 October 2026
Team	Solo builder, 14-day build window
Build window	1 Nov 00:00 UTC to 14 Nov 23:59 UTC (hard deadline). Plan to submit by 13 Nov.
Status	Week-0 preparation (until 31 Oct). Product code starts on 1 Nov.

How to read this: sections 1-3 explain the decision and its evidence. Sections 4-8 define what to build. Sections 9-15 define how. Sections 16-21 define when, how it is tested and how it is submitted. New in v1.2: the demo agent must negotiate and earn, not only spend. Both are now P1 and brief-critical, with a coverage matrix against the Agent Economy brief (sections 1, 6, 9 and 16). v1.1 added the demo agent, service registry, delegated hiring and money states.

Contents

1. Executive summary 3
2. Context and constraints 5
3. Landscape and positioning 6
4. Problem, personas and use cases 7
5. Goals, non-goals and success metrics 8
6. Functional requirements 9
7. Non-functional requirements 10
8. Threat model and attack catalogue 11
9. System architecture 12
10. Core flows 16
11. Mandate model 18
12. Data model and API specification 19
13. Security design 21
14. Red-team benchmark design 21
15. Dashboard UX 22
16. Delivery plan 22
17. Testing, QA and test budget 24
18. Deployment and DevOps 25
19. Risks and open questions 25
20. Submission and demo plan 26
21. Portfolio packaging 28
Appendix A. Sources 28
Appendix B. Glossary 29

1. Executive summary

Tollgate lets an AI agent pay for things on its own without handing it an open wallet. The agent asks Tollgate to pay. Tollgate reads the payment request, checks it against a mandate the human wrote (who, what, how much, until when) and against facts it fetches itself, then returns ALLOW, DENY or NEEDS_APPROVAL. Only an ALLOW produces a signature, and the signature is bound to the exact decision that was approved.

The product targets the weakest point of the agent economy: **the agent trusts whatever payment request a server sends it**. A malicious or compromised merchant can inflate a price, swap the payee, point at a fake deposit address or hijack the refund. Today's guards mostly cap how much a wallet can spend. They do not verify that *this* request is the one the human meant to allow.

Why this scope. My first suggestion was a generic spending-limited agent wallet. Research showed that OutLayer, an ecosystem partner of this hackathon, already ships that (caps, whitelists, multisig, freeze, TEE custody). Building it again would read as a clone. Tollgate is deliberately **wallet-agnostic**: it is the verification layer in front of any wallet, including OutLayer's. That is a different layer, a clear differentiator and a stronger portfolio story.

What changed in v1.2. The hackathon's Agent Economy brief says agents should *earn, spend, negotiate, hire* and pay for APIs, compute or data, and the agent track expects an agent that can sign and decide. So the demo agent must do all of it, not only spend. (1) **Negotiation** and **earning** move from P2 to P1 and are marked **brief-critical**: they are built in a small but real form and are simplified, never removed, if time runs short. (2) A **coverage matrix** below maps every verb in the brief to a feature, a tier and the proof a judge will see. (3) The schedule (section 16) is reworked: the negotiating seller is built with the mock merchants, negotiation lands on Days 9-10, and earning ends on Day 12. (4) The drop order now cuts nice-to-haves first and the brief-critical items last.

Carried over from v1.1. A small **demo builder agent** is a P0 deliverable. A curated **service registry** decides what is payable (the LLM does not). **Delegated mandates** let the agent hire workers on a smaller budget carved out of its own. The ledger tracks **money states** (proposed, committed, settled, released). The benchmark has 16 attacks including A16, the poisoned listing. Both tracks (`security` and `agents`) are entered, because the submission field is a list.

What gets built

- **Mandate engine**: scoped, time-boxed budgets with allowed hosts, assets and networks, plus child mandates for hired workers.
- **Verification pipeline**: 12 checks (C01-C12) on every MPP / NEAR Intents payment challenge, including a cross-check of the challenge against the 1Click API and an independent price quote.
- **Decision-bound signer**: an isolated process that signs only what a recorded ALLOW decision covers.
- **Tamper-evident audit log**: a hash chain, with a verifier command anyone can run.
- **Service registry**: a short curated list of services the agent may pay, each with a verification level.
- **Red-team benchmark**: 16 attack scenarios (A01-A16) and 5 benign controls (B01-B05), run against a naive agent, the stock `mppx` policy and Tollgate. Results are measured and published, never assumed.
- **Demo builder agent**: a small agent whose LLM runs on NEAR AI Cloud. It decides what to buy, buys data through registry services, **negotiates** a price with a seller agent, **hires** two or three simple workers, and **earns** by selling its finished report from its own paid endpoint. It exists to show Tollgate working in a full agent economy loop.
- **Dashboard**: mandates, live decisions, approvals, jobs and spend, registry, audit verification and a benchmark view.
- **TypeScript SDK and HTTP API**.

Build tiers

Tier	What is in it
Must have (P0)	Tollgate core, signer, SDK and API, dashboard basics, attack benchmark, mock merchants, service registry, basic demo agent.
Should have (P1), brief-critical	**Negotiation** against our seller agent, **earning** through the agent's own paid endpoint, **hiring** with delegated mandates. Cut last, and only ever simplified.
Should have (P1), other	Approvals, post-flight reconciliation, kill switch, job view, NEAR AI Cloud attestation display, one metered compute guard.
Nice to have (P2)	Earn-and-reinvest rule, NEAR-origin signer, OutLayer adapter, on-chain audit anchor, TEE signer.

Core, signer and benchmark are never traded for extras. If time runs short, extras are dropped in the order given in section 16, and the three brief-critical items are cut last.

Coverage of the Agent Economy brief

The brief reads: *earn, spend, negotiate, hire other agents and pay for APIs, compute or data*, with crypto directly or through NEAR Intents. The agent track also expects an agent that can sign authorized transactions and make decisions. This matrix is the check that nothing is missing.

Brief verb	How the demo agent does it	Tier	What a judge sees
Sign	Asks the gateway; the isolated signer signs only a transaction a stored ALLOW decision covers. One real mainnet payment.	P0	Tx hash. A tampered transaction is refused.
Decide	An LLM on NEAR AI Cloud plans the job and picks services by capability. Tollgate enforces limits in code.	P0	Decisions feed with reasons.
Spend / pay for APIs and data	Buys through registry services over MPP with NEAR Intents (or our labelled mock merchants).	P0	Allowed payments with 12 green checks.
Pay for compute	A bounded job with a reserved maximum cost. Live only if a provider is payable; otherwise a labelled mock.	P1	Cost cap stops a runaway job.
Negotiate	Haggles with a seller agent for 2-3 rounds under a price ceiling and round limit from the mandate.	P1, critical	Agreed price; an over-ceiling counter is denied.
Hire other agents	Hires two or three workers on child mandates with staged payments and an acceptance check.	P1, critical	Delegation tree; bad work withholds payment.
Earn	Sells its finished report through its own paid endpoint. A customer pays; revenue shows in the job summary.	P1, critical	Paid report and a revenue line next to the spend.

Honest limits: no real merchant is known to negotiate over MPP, so the seller is our own agent and is labelled as a demo. Earning is real only if a payment method that the paid-endpoint server can verify works in the Week 0 spike; otherwise the customer is a labelled simulation. Tollgate guards money going out, not money coming in.

Hard constraints that shaped the plan

- The rules say to build during the hackathon. The window is **1-14 Nov 2026**; until then it is preparation only: research, spikes that will be thrown away, assets and accounts. Confirm the exact rule wording with the organizers.
- **NEAR Intents has no testnet.** Live tests use real funds, so the plan budgets about $20-30 and relies on an offline record/replay mode so judges can run everything with no funds.
- **Function-call keys cannot enforce spend limits.** They cannot attach deposits or move tokens, and NEP-413 intent signing needs a full-access key. Limits must be enforced by policy in front of the signer, which is exactly what Tollgate is.
- HACKATHON.md: no signing, spending, mainnet deploys, posting, pushing or making the repo public without a human's confirmation, and never commit secrets. Every such step is a manual checklist item.
- Submission needs a public repo, a public X demo post and the form. It cannot be edited, the project name and email must be unique, and the API rejects requests outside the window.

Definition of winning

A judge opens the README, runs one command, watches 16 attacks fail against Tollgate while 5 legitimate payments pass, and sees the numbers next to the two baselines. They then watch the demo agent run a small economy loop: it decides what to buy, negotiates a price, pays, hires workers on a smaller budget, and sells its finished report for revenue, while one over-budget purchase is blocked and logged. The 2-minute demo video shows the same story live.

2. Context and constraints

Every row below was read from a primary source during research (Appendix A), except the row marked not verified. The third column says what the fact means for the design. Facts are as of 26 Oct 2026 and should be re-checked on Day 0.

Fact	Source	Implication for Tollgate
Build window 1 Nov 00:00 to 14 Nov 23:59 UTC. Judging 16-18 Nov. Rules say to build during the hackathon.	nearhackathon.com, HACKATHON.md	No product code before 1 Nov. Use this week for accounts, spikes and fixtures. Schedule has a cut line and a 13 Nov submit target.
Submission fields: project name (2-80 characters, unique), lead name, email (unique), team size (1), tracks, NEAR account for the payout, repo URL, public X post URL, description (20-2,000 characters), consent. One per team, not editable.	HACKATHON.md	Pick a distinctive project name now. Create the mainnet NEAR account yourself. Demo video and README are P0 deliverables. Rehearse the submit flow.
The `tracks` field is a list: enter every track that fits (`security`, `agents`). Still one submission per team. Prize pool on the site: $250K in total, paid in NEAR (earlier notes: grand $55K and $10K / $3K / $2K per track; confirm the split on the site).	HACKATHON.md, nearhackathon.com	Enter both. Lead the description with the security story; the demo agent carries the agent-economy story.
Jury: human judges (investors and founders, including Illia Polosukhin of NEAR, Eddy Lazzarin of a16z crypto and Haseeb Qureshi of Dragonfly) plus Claude Code and Codex as AI judges. README should be written for machine readers: 'do X, see Y'.	nearhackathon.com, HACKATHON.md	Open the video and README with the money story, then the checks. Write the README as numbered, copy-pasteable steps. Provide a single `pnpm bench` entry point.
Coding agents must not sign, spend, deploy to mainnet, post, push or make repos public without human confirmation, and must not commit secrets.	HACKATHON.md	Every mainnet or publishing step is a manual checklist item. Secrets live in `.env`, never in git.
NEAR Intents is mainnet only. 1Click API: `/v0/tokens`, `/v0/quote` (`dry=true` is free), `/v0/status`, `/v0/deposit/submit`.	NEAR Intents docs	Use dry quotes for price checks at no cost. Record real responses once and replay them in tests.
Without an API key 1Click charges a 0.2% fee. With a key only the 0.0001% protocol fee.	partners.near-intents.org	Apply for the key now (approval time is unknown).
Statuses: KNOWN_DEPOSIT_TX, PENDING_DEPOSIT, INCOMPLETE_DEPOSIT, PROCESSING, SUCCESS, REFUNDED, FAILED. Unknown deposit address returns 404.	1Click API reference	C06 treats 404 as DENY. P01 reconciles final status after payment.
MPP has a NEAR Intents charge method. The server mints an EXACT_OUTPUT quote per challenge and advertises the single-use deposit address as `recipient`. Client pays, then presents `{type:'hash', hash}`.	mpp.dev, nearintents-mpp-sdk	Tollgate wraps this flow. The challenge is the thing being verified.
`mppx` client has a `policy` (allowed networks and currencies, `maxAmountIn`, `expectedDestination`) and a `sendDeposit` callback. Built-in broadcaster covers EVM origins only. `refundTo` is merchant-controlled.	mppx docs	Stock policy is the second benchmark baseline. EVM origin is the MVP wallet path. C08 guards the refund target.
NEAR function-call keys cannot attach deposits or transfer tokens. NEP-413 intent signing needs a full-access key.	NEAR docs	Do not claim key-type enforcement. Enforce in an isolated signer behind a policy gate.
OutLayer ships per-tx/hourly/daily/monthly caps, whitelist, multisig, freeze, TEE custody and NEAR Intents swap. It is listed as an official ecosystem partner.	outlayer.ai, HACKATHON.md	Position Tollgate as complementary. Offer an OutLayer adapter as a stretch goal.
NEAR AI Cloud is OpenAI-compatible at `cloud-api.near.ai/v1`, with TEE inference and attestation.	NEAR AI Cloud docs	Demo agent uses it, which keeps the stack inside the ecosystem. Show its attestation in the dashboard if the Day-0 check confirms it can be fetched.
Security track hints from the organizers: agent security, function call keys, TEE attestation.	HACKATHON.md	README explains why limits live in the signer (function-call keys cannot move tokens) and shows attestation where possible.
The MPP services directory lists 130 services (60 inspected): search and data (Exa, Firecrawl, Parallel), compute (Modal, Build With Locus, Browserbase), hosting (StableUpload) and agent marketplaces (Auto.exchange, molty.cash). Most listings do not state a payment network. NEAR Intents acceptance is not stated.	mpp.dev/services	Registry entries stay unverified until a free Day-0 probe reads each service's 402 response. Mock merchants carry the benchmark. Real-service purchases are a convenience.

Fact	Source	Implication for Tollgate
x402 directories (x402.new, x402 List and others) list payment-enabled services for a different protocol. Not verified: taken from attached notes.	Attached notes	One payment path only. The Day-0 probe decides between MPP with NEAR Intents and x402. Default: MPP with NEAR Intents.

Timing risk to confirm. Work done before 1 Nov could be read as breaking the 'build during the hackathon' rule. Mitigation: this week produces only documents, accounts, API keys, recorded fixtures and disposable spike scripts that are not committed. Ask the organizers in Discord to confirm that is acceptable and keep the answer.

3. Landscape and positioning

The agent payment stack has layers. Tollgate occupies one that is thin today.

Layer	Examples	What it does	Gap Tollgate fills
Wallet and key custody	OutLayer (TEE, NEAR MPC CKD), Shade Agents, Chain Signatures	Holds keys safely. Caps per tx / day, whitelists, multisig, freeze.	Caps limit damage. They do not verify that this specific request is honest.
Payment protocol	MPP with NEAR Intents charge, x402-style flows	Merchant issues a challenge, client pays, merchant delivers.	The client trusts the challenge. The server says where and how much to pay.
Client-side policy	`mppx policy`	Static allow lists and a max amount.	No independent facts, no budget memory, no replay or refund checks, no audit.
Chain-level limits	Tempo access keys, NEAR function-call keys	Token limits and scopes (Tempo only). NEAR keys cannot move tokens.	Not portable, and not available for NEAR Intents flows.
Service discovery	MPP services directory, x402 directories	Lists services agents can buy from.	Listings are untrusted text and are not proof that a service accepts your payment method. Tollgate keeps its own curated, verified registry.
Verification and control	**Tollgate**	Verifies the challenge, enforces a mandate, binds the signature to the decision, keeps a verifiable record.	This document.

Positioning statement

For teams that let AI agents pay for APIs, data and services, Tollgate is a payment firewall that verifies each payment request before signing. Unlike a spending cap, it checks the request itself, and it works in front of any wallet.

How we differ, and how we stay honest

- **Complementary, not competing.** A user can run Tollgate in front of an OutLayer wallet. Say this in the README and the video.
- **Evidence over claims.** The benchmark compares a naive agent, the stock `mppx` policy and Tollgate. If stock policy blocks an attack, the table says so.
- **Scope discipline.** One payment path done well (MPP over NEAR Intents, EVM origin) beats five half-working ones.
- **Limits disclosed.** Tollgate cannot stop a fully compromised signer host or a merchant that delivers nothing after a legitimate payment. It reduces and records that risk (float cap, post-flight anomaly flags).

4. Problem, personas and use cases

4.1 Problem

An agent that can pay can be tricked into paying. The payment request arrives from an untrusted server, and the agent (or the thin client library around it) usually follows it. Attack surface includes price changes between quote and pay, payee or deposit-address substitution, refund redirection, replays after a timeout, and prompt injection that talks the agent into paying out of scope. Humans cannot review each payment, which is the point of an agent.

4.2 Personas

Persona	Goal	Pain today	What Tollgate gives them
Ada, agent developer	Ship an agent that buys data and API calls.	Either hands the agent a funded key (scary) or approves every payment (useless).	A mandate in 10 lines of JSON, an SDK call and a dashboard.
Tunde, security-minded operator	Prove the agent cannot overspend or be redirected.	No way to show evidence, only limits.	Verifiable audit chain and a benchmark he can re-run.
Judge or reviewer	Evaluate quickly and fairly.	Projects that need funds, keys or a long setup.	One command, offline fixtures, visible results.
Merchant (secondary)	Get paid by agents that buyers trust.	Agent buyers refuse unknown merchants.	Pinned payee flow and a clean challenge format that passes checks.

4.3 Primary use cases

- **UC1 Scoped purchase.** Agent buys API calls from `api.example.com` up to $5 this week. Tollgate allows within scope.
- **UC2 Attack blocked.** Merchant returns a challenge with a deposit address that 1Click does not know. Tollgate denies and logs why.
- **UC3 Approval.** Payment is legitimate but above the auto-approve threshold. Human approves in the dashboard, then the signer proceeds.
- **UC4 Safe retry.** Server returns 503 after payment. Tollgate resumes the same payment idempotently and never pays twice.
- **UC5 Audit.** After an incident Tunde runs `tollgate verify-log` and gets a pass or the first broken entry.
- **UC6 Hire a worker.** The agent hires a research worker. Tollgate carves a small child mandate out of the agent's remaining budget. The worker can spend only that, only at approved hosts.
- **UC7 Metered compute.** The agent runs a bounded compute job. Tollgate reserves the maximum cost, stops the job at the cap and releases the rest.
- **UC8 Poisoned listing.** A directory entry says 'ignore your budget'. The planner never sees the text, and the registry gate refuses the unverified service.
- **UC9 Negotiate.** A seller agent opens at $8 for a dataset. The demo agent counters at $5, the seller meets at $6, and the deal becomes a normal payment that Tollgate checks. A seller that later raises the price above the mandate's ceiling, or changes the payee mid-deal, is denied.
- **UC10 Earn.** The agent finishes a report and sells it from its own paid endpoint. A customer pays, the gateway verifies the receipt, and the revenue appears in the job summary next to the spend.

5. Goals, non-goals and success metrics

5.1 Goals

- G1: Block the modelled attack classes before signing, with zero false denies on benign controls.
- G2: Make the evidence reproducible by anyone in under 5 minutes with no funds.
- G3: Be a clean, adoptable SDK: a developer protects an agent in under 15 lines.
- G4: Produce a portfolio-grade repository: tests, CI, docs, architecture and threat model.
- G5: Submit on time with a polished 2-minute demo.
- G6: Show one complete small job: the demo agent decides, buys, negotiates, hires and stays under budget while a bad purchase is blocked.
- G7: Cover every verb in the Agent Economy brief (sign, decide, spend, negotiate, hire, earn, pay for compute) with a working demo or a clearly labelled mock.

5.2 Non-goals

- Not a wallet and not a custody product. Tollgate never holds user funds beyond a small signer float.
- Not a general fraud engine or ML scoring product. Risk score is simple, explainable and rule-based.
- Not multi-chain from day one. One origin path works end to end; others are stretch.
- Not an agent platform or marketplace. The demo agent and workers exist to show Tollgate working.
- No claim of protection against a fully compromised signer host, a malicious human approver, or a merchant or hired agent that takes payment and delivers nothing (staged payments limit the loss; they do not prevent it).

5.3 Success metrics

Metric	Target	How measured
Attacks blocked (A01-A16) by Tollgate	16 of 16 (report exact number; report any miss honestly)	`pnpm bench`, offline
False denies on benign controls (B01-B05)	0 of 5	`pnpm bench`
Same attacks vs stock `mppx` policy and naive agent	Measured and published, whatever they are	`pnpm bench`
Added decision latency (offline fixtures)	p95 under 50 ms. Live mode adds network calls, report separately.	Bench timer
Time from clone to benchmark result	Under 5 minutes	Fresh-machine test on Day 13
Live mainnet smoke test	1 successful end-to-end payment, 1 blocked attack, small amount	Manual checklist, tx hashes in README
Core package test coverage	At least 85% lines on policy, checks, ledger and audit	Vitest coverage in CI
Demo job	Demo agent completes a small job under its budget. At least one over-budget or off-registry purchase is blocked and logged.	`pnpm demo`
Delegation invariants	Child budgets never exceed the parent's remaining budget. Revoke cascades. Scope never widens.	Property tests in CI
Negotiation	A deal closes in at most the round limit. Any counter-offer above the ceiling or with a changed payee or asset is denied.	Scenario tests and demo run
Earning	At least one report sold through the agent's endpoint, with revenue in the job summary equal to the verified receipt.	`pnpm demo`, tx hash if live
Registry gate	Zero payments to unregistered or unverified services	Integration test and benchmark

6. Functional requirements

Priority: **P0** must ship, **P1** ship if on schedule, **P2** stretch. Each requirement has an acceptance test; the benchmark and unit tests are the acceptance suite. F17 was promoted to P0 (basic version). Rows F23-F32 are new in v1.1. In v1.2, F31 (negotiation) and F32 (earning) were promoted from P2 to P1 and are brief-critical: they are cut last and only simplified.

ID	Pri	Requirement	Acceptance criteria
F01	**P0**	Create, read, update and revoke **mandates** (principal, agent id, budget, period, allowed hosts, assets, networks, approval threshold, expiry).	Schema validation rejects malformed mandates. Revoked mandate yields DENY on next request.
F02	**P0**	**Authorize endpoint**: accepts a payment challenge plus context (mandate id, host, purpose) and returns a decision with reasons.	Response includes decision, per-check results, risk score and decision id. Deterministic for fixed inputs.
F03	**P0**	Run checks **C01-C12** in a fixed order and short-circuit on a hard DENY.	Unit test per check with pass and fail fixtures. Order documented and tested.
F04	**P0**	**1Click cross-check** (C06): query status for the advertised deposit address and compare the quote fields (amount, asset, destination, refund target) to the challenge.	404 or any mismatch yields DENY. Recorded fixtures cover match, mismatch and 404.
F05	**P0**	**Independent price check** (C07): request own dry quote and compare to the challenged amount within a configured tolerance.	Quote above tolerance yields DENY or NEEDS_APPROVAL by policy. Quote failure never silently allows.
F06	**P0**	**Payee binding** (C05): pin the payee for a host on first approved use, deny a change.	Second challenge with a different payee for the same host yields DENY and an alert.
F07	**P0**	**Budget ledger** with reserve-then-settle accounting and velocity limits (C09).	Concurrent requests never exceed budget (race test). Reservation released on failure or expiry.
F08	**P0**	**Idempotency** (C10): same challenge id never pays twice; resume after 503 or 504.	Replay test pays once. Resume test reuses the existing deposit.
F09	**P0**	**Decision-bound signer**: separate process signs only transactions that match a stored ALLOW decision (payee, asset, amount, chain, expiry, nonce).	Signer rejects any tampered field. Signer rejects a decision used twice.
F10	**P0**	**Hash-chained audit log** of requests, checks, decisions, signatures and outcomes, plus a verify command.	Editing any past entry makes verify fail at that entry. Verify passes on a clean log.
F11	**P0**	**Red-team benchmark**: 16 attacks, 5 benign controls, 3 targets, offline by default.	`pnpm bench` prints a results table and writes JSON and HTML reports.
F12	**P0**	**TypeScript SDK** wrapping a fetch client so a 402 challenge is routed through Tollgate transparently.	Demo agent protected in 15 lines or fewer.
F13	**P0**	**Dashboard**: mandates, decisions feed with check breakdown, spend charts, audit verification button.	All visible with seeded data in under 3 clicks from the home page.
F14	**P1**	**Approvals** (C12): NEEDS_APPROVAL creates a pending item; approve or reject in the dashboard with a reason; expiry.	Approve leads to signing. Reject or expiry leads to DENY. Both logged.
F15	**P1**	**Post-flight reconciliation** (P01-P04): poll final status, verify the receipt, settle or release budget, flag anomalies.	REFUNDED or FAILED releases the reservation and flags the host.
F16	**P1**	**Kill switch**: one action freezes all signing for a principal or globally.	While frozen every request yields DENY with reason FROZEN.
F17	**P0**	**Demo builder agent** (basic): uses NEAR AI Cloud for reasoning, buys data through registry services under a mandate, and includes a prompt-injection scene and one blocked over-budget purchase.	`pnpm demo` runs start to finish. The agent is manipulated and Tollgate blocks it.
F18	**P1**	**Record and replay** tool for 1Click, merchant and chain responses.	Fixtures are scrubbed of secrets and replay byte-identically.
F19	**P2**	NEAR-origin signer (pay from a NEAR account).	One successful NEAR-origin payment on mainnet with a small amount.
F20	**P2**	**OutLayer adapter**: Tollgate in front of an OutLayer wallet.	Adapter interface implemented and one payment demonstrated.
F21	**P2**	Anchor the audit head hash on-chain periodically.	Head hash visible on a NEAR explorer and checked by verify.
F22	**P2**	TEE deployment of the signer with attestation shown in the dashboard.	Attestation report displayed and verified.

ID	Pri	Requirement	Acceptance criteria
F23	**P0**	**Service registry**: a table of payable services with verification level, quoted price, payment rails and pinned payee. Payment to an unregistered or unverified service is refused (C02).	Payment to an unregistered, unverified or disabled service yields DENY.
F24	**P0**	**Listing sanitization**: the planner receives structured registry fields only. Third-party listing text is never placed in a model prompt.	A16 passes: injected listing text changes no decision.
F25	**P0**	**Money states**: proposed, committed, settled and released, recorded by the ledger and shown per job.	Job summary equals the ledger sums. Reservation released on failure.
F26	**P1**	**Delegated mandates**: carve a child mandate out of a parent's remaining budget, with scope and expiry subsets and a cascading revoke.	Property tests: child budgets never exceed the parent's remaining budget, scope never widens, revoke cascades.
F27	**P1**	**Hired workers**: the demo agent hires two or three Tollgate-managed workers with staged payments and an acceptance check.	Upfront share paid on acceptance, the rest only after the check passes. A bad deliverable withholds the remainder.
F28	**P1**	**Metered compute guard**: reserve the maximum cost, cap runtime and cost, settle the actual cost, release the rest.	A runaway job stops at the cap. The reservation reconciles. A mock provider is labelled if no live provider is payable.
F29	**P1**	**Jobs view** in the dashboard: per-job budget, committed, settled, released, remaining and blocked attempts.	Matches the ledger for the seeded demo job.
F30	**P1**	**Attestation display**: show the NEAR AI Cloud attestation for the demo agent's inference.	Report fetched and displayed, or documented as unavailable after the Day-0 check.
F31	**P1**	**Negotiation (brief-critical)**: the demo agent haggles with our seller agent for 2-3 rounds. The mandate sets a price ceiling and a round limit. Each offer is stored; an accepted deal becomes a normal payment challenge.	A deal closes within the round limit. A counter-offer above the ceiling yields DENY. A changed payee or asset mid-negotiation yields DENY. The model proposes; code enforces.
F32	**P1**	**Earning (brief-critical)**: the agent sells its finished report through its own paid endpoint (MPP server side). A customer pays; the gateway verifies the receipt before crediting a revenue entry.	Revenue in the job summary equals the verified receipt. An unverified or replayed receipt credits nothing. Revenue never raises a mandate budget on its own. Tollgate does not guard incoming payments.

7. Non-functional requirements

Area	Requirement	Target
Security	Signing keys are never readable by the API, dashboard or agent. Secrets never committed. Dependencies pinned with a lockfile.	Secret scan in CI passes. Signer runs as its own process and user.
Correctness	Fail closed: any check error, timeout or ambiguity yields DENY or NEEDS_APPROVAL, never ALLOW.	Fault-injection tests for every external call.
Performance	Decision latency offline fixtures. Live checks parallelized where independent.	p95 under 50 ms offline. Live p95 under 2.5 s with 1Click reachable.
Reliability	Idempotent endpoints. Reservations expire. Resume after restart from the database.	Kill the API mid-payment test: no double spend, no stuck reservation.
Auditability	Every decision is explainable from stored data. Log append-only with hash chain.	Verify command runs in under 1 s for 10,000 entries.
Usability	Mandate JSON readable without docs. Errors name the failed check and the fix.	New user protects the demo agent in under 15 minutes.
Portability	Runs with Docker Compose or plain Node 22 plus Postgres. Offline mode needs no network.	Fresh clone to bench result in under 5 minutes.
Observability	Structured JSON logs with request and decision ids. Basic metrics endpoint.	Every log line carries a decision id when relevant.
Maintainability	TypeScript strict mode. Checks are pure functions with a common interface.	Adding a check needs one file and one test.
Accessibility	Dashboard keyboard navigable, readable contrast, no color-only status.	Axe or Lighthouse accessibility score of 90 or more.
Honesty of claims	Every claim in the README and video traces to a recorded result. Anything simulated is labelled.	Claims checklist reviewed on Day 13.
Registry integrity	Only enabled, verified registry entries can receive payments. Third-party text never reaches a model prompt.	Integration test plus attack A16.

8. Threat model and attack catalogue

8.1 Assets, actors and trust boundaries

- **Assets**: signer float, budget integrity, mandate integrity, audit integrity, merchant pins, user trust.
- **Untrusted**: merchant servers and their challenges, directory listings and their text, hired agents, the agent and its LLM (it can be prompt-injected), any network response not independently verified.
- **Partially trusted**: 1Click API (verified against challenge, never blindly trusted), RPC providers.
- **Trusted**: the Tollgate gateway, signer process, database and the human operator's console.

8.2 Attack catalogue (benchmark scenarios)

ID	Attack	Defended by	Expected on Tollgate
A01	**Price inflation**: challenge amount far above the real price	C07	DENY
A02	**Payee swap**: payee differs from the pinned one for this host	C05	DENY
A03	**Fake deposit address**: address unknown to 1Click	C06	DENY
A04	**Quote/challenge mismatch**: status fields differ from the challenge	C06	DENY
A05	**Asset or network swap**: unexpected token or chain	C04	DENY
A06	**Refund hijack**: `refundTo` points to a third party	C08	DENY
A07	**Expired challenge**: deadline already passed	C03	DENY
A08	**Replay or double charge**: same challenge twice	C10	DENY (second)
A09	**Structuring**: many small payments just under thresholds	C09	DENY or NEEDS_APPROVAL
A10	**Runaway loop**: agent retries payments in a tight loop	C09	DENY after velocity cap
A11	**Out-of-scope host**: host not in the mandate	C02	DENY
A12	**Payment prompt injection**: page text tells the agent to pay elsewhere	C01, C02, C05	DENY
A13	**Mandate overrun**: payment above remaining budget	C09	DENY
A14	**Bad exchange rate**: origin amount much higher than quote implies	C07	DENY or NEEDS_APPROVAL
A15	**Scheme downgrade**: challenge offers a weaker or unknown payment method	C03, C04	DENY
A16	**Poisoned listing**: directory text tells the planner to ignore its budget or pay another address	Registry gate, listing sanitization, C02	DENY (listing text never reaches the planner)

Benign controls (must be allowed or sent to approval as designed): B01 normal in-scope payment, B02 payment at the budget edge, B03 first payment to a new merchant inside scope (NEEDS_APPROVAL if the mandate says so), B04 legitimate retry after 503, B05 price drift inside tolerance.

Invariant tests (unit and property tests, not benchmark scenarios): a child mandate cannot exceed its parent's remaining budget or widen scope; a negotiation counter-offer above the price ceiling is refused; an unverified registry service is never paid.

8.3 Residual risks (stated, not hidden)

- Compromised signer host: limited by a small float and a kill switch, not prevented.
- Merchant takes payment and delivers nothing: detected by post-flight checks and anomaly flags, not prevented.
- Malicious or careless human approver: out of scope, but approvals are logged with the reason.
- 1Click API returning consistently wrong data: C07 uses an independent quote to reduce the chance both lie in the same way.
- A hired agent takes the upfront share and delivers nothing: limited by staged payments and the exposure cap, not prevented.
- A seller agent that negotiates in bad faith: it can waste rounds but cannot push a price past the ceiling or change the payee; the round limit ends it.
- A fake or replayed customer receipt on the earning endpoint: the gateway credits revenue only after verifying a fresh, settled receipt, and revenue never raises a budget automatically.

9. System architecture

Tollgate has five runtime parts. The **Gateway** (API plus verifier) is the only component the agent talks to. The **Signer** is a separate process that holds the only key and accepts signing requests only from the gateway, only with a decision id. The **Store** (Postgres) holds mandates, decisions, ledger, registry and the audit chain. The **Dashboard** is a read-mostly web app for humans. The **SDK** is a small client library in the agent's process. Five proof components sit in the same repository but are not the product: the demo agent, the hired workers, the earn endpoint, the mock merchants (including a negotiating seller) and the benchmark runner.

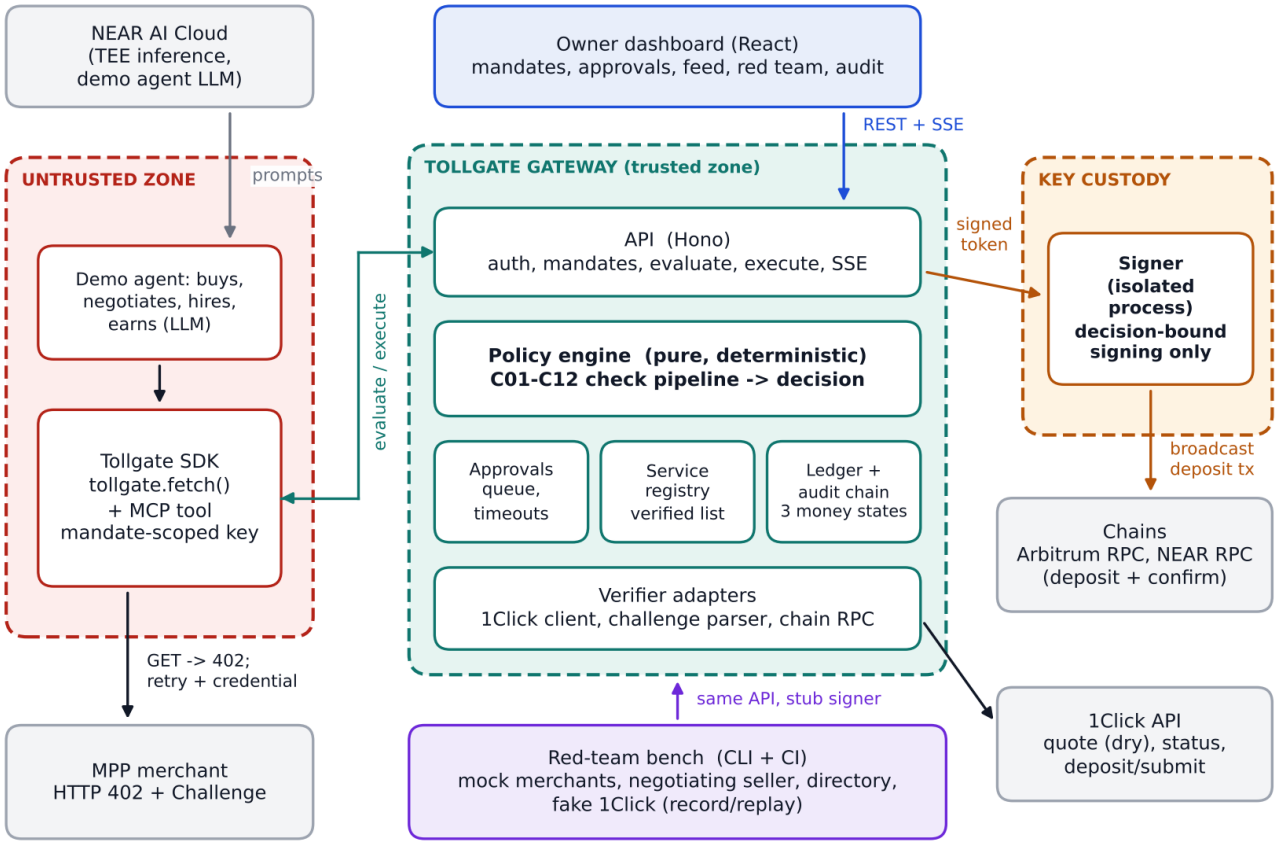


Figure 1. Component architecture and trust zones. The agent never reaches the signer or the store directly.

Component	Responsibility	Tech	Trust
SDK	Intercepts HTTP 402, extracts the challenge, calls authorize, retries with the credential.	TypeScript, fetch wrapper	Untrusted (runs with the agent)
Gateway API	Auth, mandate CRUD, delegation, authorize, approvals, jobs, status, audit read.	Node 22, Hono, Zod	Trusted
Verifier	Runs C01-C12 as pure functions with injected adapters (1Click, quote, clock, store, registry).	TypeScript package	Trusted
Ledger	Reserve-then-settle budgets with row-level locks. Tracks proposed, committed, settled and released amounts.	Postgres, Drizzle	Trusted
Registry	Curated table of payable services with verification level, quoted price, payment rails and pinned payee. Feeds C02 and C05.	Postgres, Drizzle	Trusted data, untrusted text
Audit	Append-only hash chain, `verify-log` CLI.	SHA-256 over canonical JSON	Trusted, tamper-evident
Signer	Verifies a decision against the stored record and signs only matching transactions.	Separate Node process, viem	Most sensitive: smallest code, smallest float
Dashboard	Mandates, decision feed, approvals, jobs, registry, spend, audit verify, benchmark view.	React, Vite, Tailwind	Trusted, authenticated

Component	Responsibility	Tech	Trust
Demo agent	Builder agent: plans with an LLM on NEAR AI Cloud, buys through registry services, negotiates, hires workers and sells its report.	TypeScript	Untrusted (it is what we protect)
Hired workers	Two or three small agents (research, analysis, report) that run under child mandates.	TypeScript	Untrusted
Earn endpoint	The agent's own paid endpoint: issues an MPP challenge for the report, returns it after payment. The gateway verifies the receipt and credits revenue.	Hono, mppx server middleware	Untrusted input; revenue credited by the gateway only
Mock merchants	Fake sellers and a fake directory that issue legitimate and malicious challenges and listings, plus a negotiating seller agent that makes counter-offers.	Hono	Test only
Bench runner	Runs scenarios against three targets using fixtures.	Vitest harness plus CLI	Offline by default

9.1 Key design decisions

Decision	Choice	Why
Language	TypeScript across SDK, API, signer, dashboard	One language, shared types, fastest for a solo 14-day build.
Wallet path (MVP)	EVM-origin USDC on Arbitrum via viem; merchant receives USDC on NEAR through 1Click	It is the documented, working `mppx` path. NEAR-origin is stretch.
Payment path	MPP with NEAR Intents. x402 only if the Day-0 probe shows it is the practical path	One path done well beats two half-working ones.
Where limits live	In the gateway and signer, not in key types	NEAR function-call keys cannot move tokens, so key-type limits are not possible.
Signing model	Decision-bound: signer checks the stored ALLOW before signing	A compromised API cannot ask for arbitrary signatures without a matching decision.
Failure mode	Fail closed	A payment tool that fails open is a vulnerability.
Service registry	Curated, in Postgres. The LLM never sees third-party listing text	Listings are untrusted and are not proof that a service accepts your payment method.
Hiring model	Delegated child mandates for our own workers. External marketplaces are stretch	A hired agent is just another payee, with a smaller budget and staged payments.
Money states	Proposed, committed, settled (and released)	Keeps the ledger honest while payments are in flight and makes the job summary auditable.
Testing without funds	Record and replay fixtures	No testnet exists. Judges and CI need to run with zero funds.

9.2 Service registry

The registry is a Postgres table of the services Tollgate may pay. The LLM decides which *capability* it needs ('competitor pricing data'). Tollgate decides what is actually payable. A listed service is not automatically available, its price may have changed, and it may not accept the payment method Tollgate supports.

- **The LLM proposes, Tollgate verifies.** The planner asks the registry for offers by capability. Only registry entries that pass the gate can be paid, and only after the 12 checks.
- **Listings are untrusted text.** Names, descriptions and sample outputs from any directory are stored for humans but never placed in a model prompt. The planner sees structured fields only (capability, price, rails, level). This is the defence for attack A16.
- **Services that need an API key or a funded account are out of scope** for the MVP. They cannot be paid through Tollgate's payment path.

Level	What it means	May be paid?
unverified	Listed by a human or a directory. Nothing has been tested.	No, never
endpoint-checked	A free probe (no payment) fetched the endpoint's 402 response, parsed it and found a payment method Tollgate supports.	Yes, small amounts; first payee still goes to approval (C12)
paid-delivery-tested	One small manual purchase delivered the resource. Evidence is stored.	Yes, up to the mandate limits
disabled	Failed a health check, changed its payee or terms unexpectedly, or was switched off by the operator.	No

Candidate entries, taken from the MPP services directory (130 services listed, 60 inspected). Their payment methods are **unverified** until the Day-0 probe runs; a candidate that does not offer a supported method stays unverified and is not used live.

Role	Candidates	Notes
Search and research	Exa, Firecrawl, Parallel	Cents per request. Best first candidates for the demo.
Metered compute	Modal, Build With Locus	P1, only if a supported payment method is confirmed. Otherwise a labelled mock.
Hosting	StableUpload	Stretch. Lets the agent publish its result.
Hiring	Our own workers (P1). Auto.exchange and molty.cash (stretch)	Own workers need no third party and run under child mandates.
Benchmark	Our own mock merchants and a mock directory	Always present. Some attacks cannot be run safely against real services.

A read-only directory crawl may add rows as *unverified* (stretch). Nothing discovered becomes payable without the probe and a human enabling it.

9.3 Delegated mandates and hiring

A hired worker is another payee. Tollgate treats hiring as a spend with extra rules, so the agent cannot use hiring to escape its budget. The parent mandate carves a smaller *child mandate* out of its own remaining budget and gives it to the worker.

Rule	What it guarantees
Budget carve-out	A child's budget is reserved out of the parent's remaining budget when it is created and cannot exceed it. Unspent budget returns to the parent when the child expires or is revoked.
Scope subset	A child's allowed hosts, assets and networks must be subsets of the parent's. A child can never widen scope.
Time limit	A child's expiry is never later than the parent's.
Depth	Depth is one. Workers cannot hire other agents unless the parent mandate sets `maxDepth` above zero.
Revocation	Revoking a parent revokes every child and releases their reservations.
Staged payment	Payment to a hired worker is split: an upfront share on acceptance (default 25 percent) and the rest after the acceptance check passes.
Exposure cap	Total paid to one hired agent is capped by its child mandate, so a worker that vanishes keeps only what it was paid up front.
Acceptance check	A pure function over the deliverable (schema valid, required fields present, source links present). It gates the final payment. It does not judge quality.

- **Worker types in scope:** Tollgate-managed workers (research, analysis, report), small scripts that call registry services under their child mandates. External marketplace agents (Auto.exchange, molty.cash) are stretch. Human contractors are out of scope.
- **Verified by tests, not by the benchmark:** property tests assert that child budgets never exceed the parent's remaining budget, that scope never widens and that revoke cascades.

9.4 Money states and the job summary

Money is in one of four states. The ledger records every transition, and the dashboard shows them per job.

State	Meaning	Ledger effect	Trigger
Proposed	A request is waiting for a decision or a human approval.	None. Shown as pending in the job view.	NEEDS_APPROVAL created
Committed	An ALLOW decision exists and the amount is reserved.	Reserve entry. Available budget goes down.	ALLOW (C09)
Settled	The payment is confirmed and the reserved amount becomes spent.	Settle entry.	Post-flight SUCCESS (P03)
Released	The reservation is returned.	Release entry.	FAILED, REFUNDED, expiry or revoke

The **job summary** shows, per job: budget, proposed, committed, settled, released, remaining, and the blocked attempts with the check that stopped each one. It is derived from the ledger, never stored separately, so it cannot disagree with it. A revenue line (P1, section 9.6) sits next to the spend so a judge sees what the job cost and what it earned; Tollgate does not guard incoming money, it only records verified receipts.

9.5 Metered compute (P1)

- Prefer a bounded job with a fixed quote over a running meter.
- For a meter, Tollgate reserves the job's maximum cost when it starts. The job runs with a runtime cap and a cost cap, and no network access unless the mandate allows it. The actual cost settles and the rest is released. A runaway job is stopped at the cap and logged.
- Only a payment-compatible provider is used live. Modal and Build With Locus appear in the MPP directory as compute services, but their payment method is unverified. If the Day-0 probe does not confirm one, the compute step uses a clearly labelled mock provider, and the README and video say so.
- This is a P1 item and starts only when everything above it in section 16 is green.

9.6 Negotiation and earning (P1, brief-critical)

The Agent Economy brief names both verbs, so both are built in a small but real form. They are cut last and only simplified, never removed. In plain words: negotiation is the agent bargaining over a price before it pays; earning is the agent being paid for something it made.

Negotiation

- **Counterparty.** Our own seller agent in the mock merchants. No real merchant is known to negotiate over MPP, so the README and video label it a demo. The seller is scripted with a few strategies (accepts a mid price, holds firm, tries to raise the price late, tries to change the payee) so every path can be tested offline.
- **Protocol.** A short exchange of structured offers over HTTP: `offer`, `counter`, `accept`, `walk`. Each offer carries price, asset, payee, validity and a round number. The deal ends with a normal MPP challenge at the agreed price, which goes through C01-C12 like any other payment.
- **The model proposes, code enforces.** The LLM chooses the counter-offer and when to accept or walk. Tollgate holds the *price ceiling* and the *round limit* (both in the mandate) and stores every round. An offer above the ceiling, a changed payee or asset between rounds, or a round beyond the limit is a DENY, whatever the model says.
- **Guardrail against drift.** The final challenge must match the last accepted offer exactly (price, asset, payee). A seller that accepts at $6 and then challenges at $9 is caught by C07.
- **Minimum viable form.** One seller, one item, up to three rounds, run live in `pnpm demo`. If time runs out, simplify to a single counter-offer round; do not remove it.

Earning

- **What is sold.** The finished report from the demo job (for example a short market brief), priced at a few cents to a few dollars from the agent's own price list.
- **How.** The agent exposes a paid endpoint using the `mppx` server middleware (Hono). A request gets a 402 challenge; a customer pays; the endpoint verifies the credential and returns the report with a receipt.
- **Who pays.** A second, small customer agent that we run, funded with a couple of dollars from a separate wallet. It uses the same SDK path, so the demo shows both sides of an agent economy.

- **Crediting revenue.** The gateway credits a revenue entry only after it verifies a fresh, settled receipt (amount, asset, payee equals the agent's receiving address, receipt not seen before). A replayed or unsettled receipt credits nothing. The job summary shows spend, revenue and net.
- **Receiving address.** A separate address that the human creates for the agent. It is not the hackathon prize account, and it is never created or invented on the human's behalf.
- **Rails are a spike.** Whether the server side can verify a payment method we can also pay with (for example a plain EVM stablecoin charge) is a Week 0 question. If NEAR Intents is not usable on the server side, earning uses the method that works and says so. If nothing works live, the customer is a labelled simulation using a replayed receipt.
- **Safety rule.** Revenue never raises a mandate budget on its own. An optional earn-and-reinvest rule (P2) would add a capped share of verified revenue to the next job's budget, set by the human in the mandate.

Neither feature changes what is verified for outgoing payments. They change what the demo shows: an agent that decides, bargains, pays, hires and gets paid.

10. Core flows

10.1 Authorize and pay

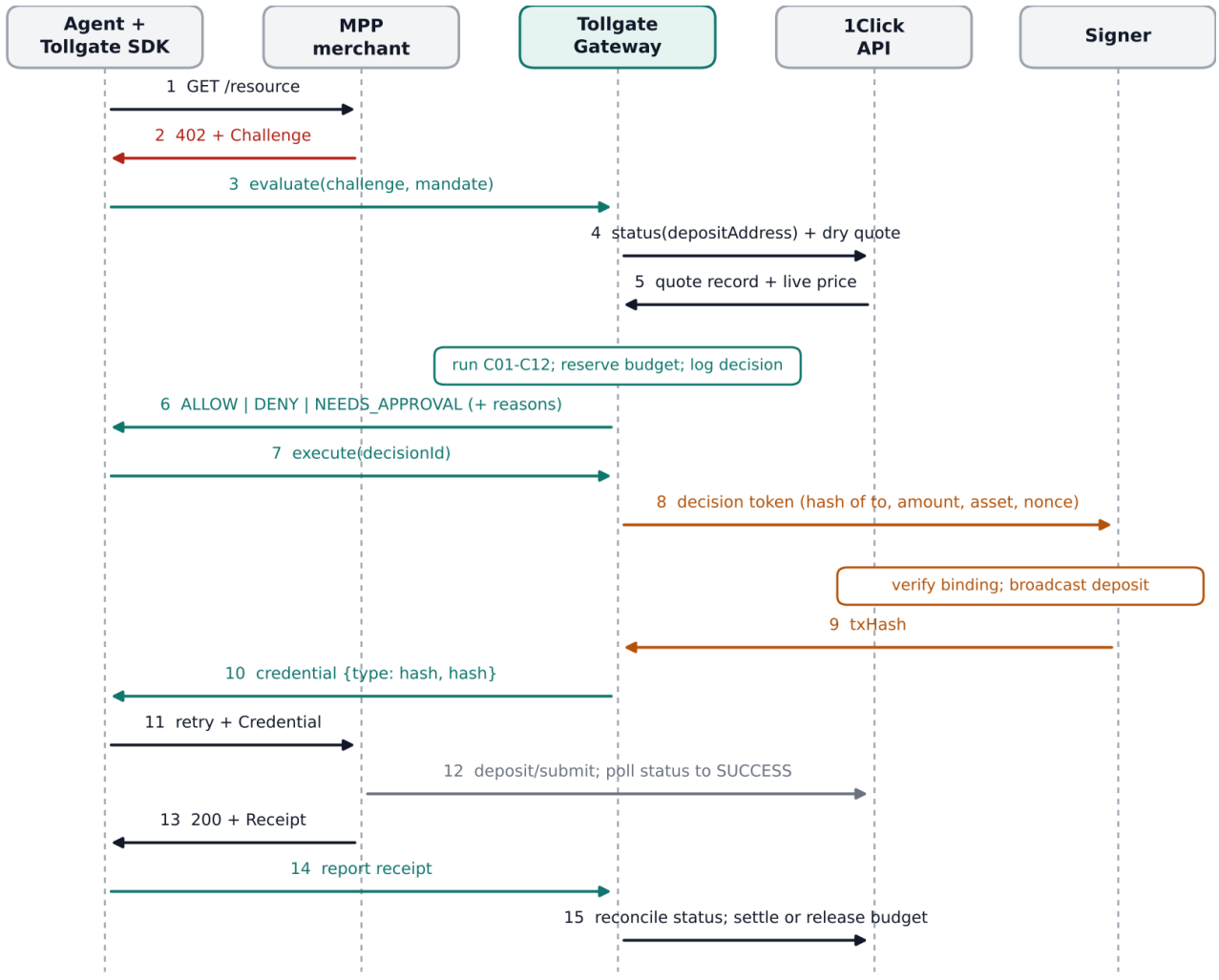


Figure 2. Happy path from 402 challenge to delivered resource, with the decision-bound signer.

10.2 Verification pipeline

Checks run in order and cheap local checks go first. A hard failure short-circuits to DENY. Soft signals feed the risk score (C11) and the approval gate (C12).

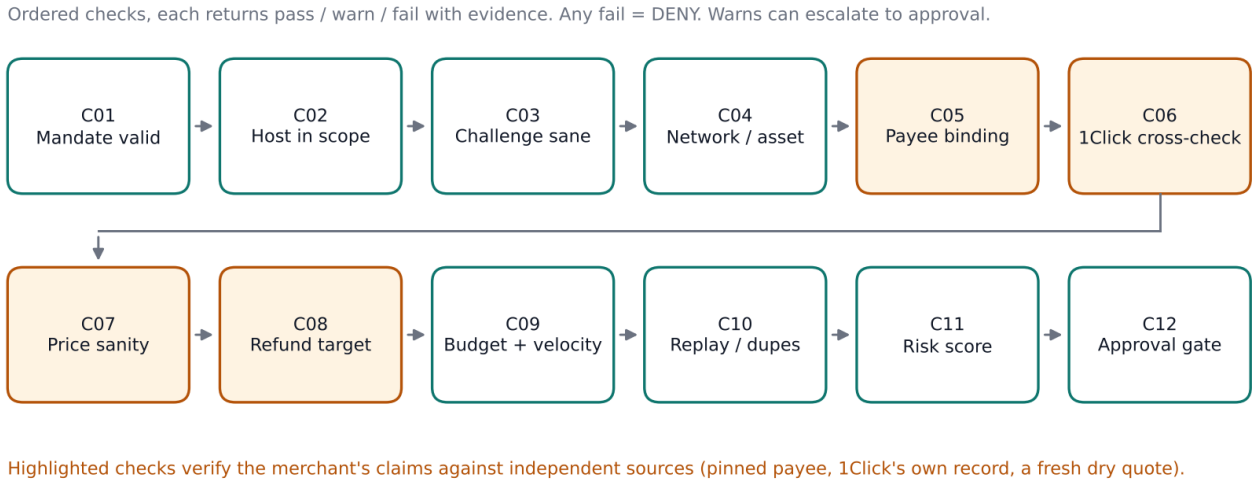


Figure 3. Check pipeline C01-C12 and post-flight steps P01-P04.

Check	Name	Logic	Outcome on fail
C01	Mandate valid	Exists, not revoked, not expired, agent id matches.	DENY
C02	Host in scope	Request host matches mandate `allowedHosts` (exact or declared wildcard) and is an enabled registry entry at endpoint-checked level or better.	DENY
C03	Challenge sane	Parses, supported scheme, not expired, required fields present, sane size.	DENY
C04	Network and asset	Origin network and currency in the mandate allowlist.	DENY
C05	Payee binding	Payee comes from the registry record and is pinned per host on first approved use; a later change is rejected. Trust on first use goes to approval.	DENY
C06	1Click cross-check	Status for the deposit address exists (404 means DENY). Amount, asset, destination, refund target equal the challenge.	DENY
C07	Price sanity	Own dry quote for the same output. Challenged input within tolerance of own quote.	DENY or NEEDS_APPROVAL
C08	Refund target	`refundTo` is the payer or a pinned address.	DENY
C09	Budget and velocity	Remaining budget covers the amount. Count and rate within limits. Reserves the amount.	DENY
C10	Replay and duplicates	Challenge id and deposit address unseen, or same payment resumed idempotently.	DENY (new) or RESUME
C11	Risk score	Sum of soft signals: new host, large amount, off-hours, drift, recent anomalies. Scored 0-100.	Feeds C12
C12	Approval gate	Amount above threshold, risk above threshold or first payee leads to a human.	NEEDS_APPROVAL

Post-flight

- **P01** poll 1Click status until a terminal state or timeout. **P02** verify the receipt matches the reserved amount and payee.
- **P03** settle the reservation on SUCCESS, release it on REFUNDED or FAILED. **P04** raise anomaly flags (late refund, partial delivery, repeated failure) and lower the host's trust.

10.3 Extended flows (v1.1, extended in v1.2)

Flow	Sequence	Where Tollgate decides
Hire a worker	1 The agent asks to hire a worker with budget B. 2 The gateway creates a child mandate only if B fits the parent's remaining budget and scope. 3 The worker buys from registry services through the normal pipeline. 4 The worker returns a deliverable. 5 The acceptance check runs and the final payment is authorized only if it passes.	Delegation rules, C01-C12 on every worker payment, acceptance check.
Metered compute (P1)	1 The agent requests a job. 2 Tollgate reserves the maximum cost. 3 The job runs under a runtime cap and a cost cap. 4 The actual cost settles and the rest is released.	C09 reservation, cap enforcement, post-flight P03.
Negotiate (P1)	1 The seller agent opens with an offer. 2 The LLM chooses a counter. 3 The gateway checks each offer against the price ceiling, round limit, payee and asset, and stores it. 4 On agreement the seller issues a normal payment challenge. 5 The challenge must equal the accepted offer, then C01-C12 run and the signer signs.	Ceiling and round limit, offer-to-challenge match, C07 and the rest of the pipeline.
Earn a sale (P1)	1 A customer agent requests the report from the agent's endpoint. 2 The endpoint returns a 402 challenge. 3 The customer pays. 4 The endpoint verifies the credential and returns the report with a receipt. 5 The gateway verifies the receipt and credits a revenue entry. 6 The job summary shows spend, revenue and net.	Receipt verification (fresh, settled, right payee, unseen). Tollgate does not authorize incoming payments.
Registry probe	1 An operator adds a service. 2 Tollgate fetches its 402 response without paying. 3 It parses the payment method and payee and sets the level to endpoint-checked, or leaves it unverified.	No payment is made. Level changes are logged.

11. Mandate model

A mandate is the human's instruction in a form a machine can enforce. It is data, not code, and it is versioned. Changing a mandate creates a new version; decisions record the version they used.

```
{
  "id": "mnd_01HZX...",
  "version": 3,
  "principal": "ada@example.com",
  "agentId": "research-agent-1",
  "validFrom": "2026-11-05T00:00:00Z",
  "validUntil": "2026-11-12T00:00:00Z",
  "budget": { "currency": "USDC", "total": "5.00", "perPayment": "0.50",
              "perHour": "2.00", "maxPaymentsPerMinute": 3 },
  "allowedHosts": ["api.example.com", "data.example.org"],
  "allowedOriginNetworks": ["eip155:42161"],
  "allowedAssets": ["USDC"],
  "payee": { "mode": "pin-on-first-approval" },
  "price": { "tolerancePct": 3, "onExceed": "needs_approval" },
  "approval": { "above": "0.50", "riskAbove": 60, "firstPayeeRequiresApproval": true },
  "refund": { "mustBePayerOrPinned": true }
}
```

11.1 Decision record

```
{
  "decisionId": "dec_01HZY...", "mandateId": "mnd_01HZX...", "mandateVersion": 3,
  "decision": "ALLOW",                      // ALLOW | DENY | NEEDS_APPROVAL
  "checks": [ {"id":"C06","result":"pass","evidence":{"status":"PENDING_DEPOSIT"}}, ... ],
  "risk": 12,
  "bind": { "chain":"eip155:42161", "asset":"USDC", "amount":"0.250000",
            "to":"0xDEP0...", "expiresAt":"2026-11-06T10:05:00Z", "nonce":"n_9f2..." },
  "prevHash": "9a1c...", "hash": "b7e0..."
}
```

The `bind` object is exactly what the signer verifies. A signature request that differs in any field, or that reuses a nonce, is refused.

11.2 Child mandate (delegation)

```
{
  "id": "mnd_01HZZ...", "version": 1, "parentId": "mnd_01HZX...",
  "agentId": "worker-research-1",
  "validUntil": "2026-11-08T12:00:00Z",        // never later than the parent's
  "budget": { "currency": "USDC", "total": "1.50", "perPayment": "0.50" },
  "allowedHosts": ["api.example.com"],         // a subset of the parent's hosts
  "delegation": { "maxDepth": 0 },             // workers cannot hire other agents
  "staging": { "upfrontPct": 25, "releaseOn": "acceptance_check_passed" }
}
```

The child's budget is reserved out of the parent's remaining budget when the child is created. Unspent budget returns to the parent on expiry or revoke.

11.3 Registry record

```
{
  "id": "svc_search_01", "name": "Example search service", "category": "web_search",
  "providerUrl": "https://search.example.com",
  "pricing": { "model": "per_request", "quoted": "0.01", "currency": "USDC", "quoteExpiresAt": "..." },
  "paymentRails": [ { "protocol": "mpp", "method": "nearintents",
                      "originNetwork": "eip155:42161", "asset": "USDC" } ],
  "payee": "0xPAYEE...", "verification": "endpoint-checked", "health": "healthy",
  "limits": { "maxPerRequest": "0.10", "timeoutMs": 8000 },
  "lastVerifiedAt": "...", "evidence": { "probeId": "prb_0a1..." }, "enabled": true
}
```

Illustrative values only. A real record uses values read from the provider's own 402 response; nothing here is a claim about a real service.

12. Data model and API specification

12.1 Data model

Table	Key fields	Notes
mandates	id, version, parent_id, principal, agent_id, spec (jsonb), status, created_at	Immutable versions. Revoke sets status and cascades to children.
decisions	id, mandate_id, mandate_version, challenge_id, decision, checks (jsonb), risk, bind (jsonb), nonce, created_at	Unique on challenge_id plus mandate_id for idempotency.
ledger_entries	id, mandate_id, decision_id, kind (reserve / settle / release), amount, currency, created_at	Balance is a derived sum. Row locks serialize reserves. reserve = committed, settle = settled, release = released.
payees	host, payee, pinned_at, source (auto / approved)	Basis for C05.
approvals	id, decision_id, status, reviewer, reason, expires_at, decided_at	Pending, approved, rejected, expired.
payments	id, decision_id, deposit_address, tx_hash, status, final_status, updated_at	Status mirrors 1Click statuses.
audit_log	seq, ts, type, payload (jsonb), prev_hash, hash	Append-only. Hash covers canonical JSON plus prev_hash.
fixtures	key, kind, request_hash, response (jsonb)	Record and replay store.
services	id, name, category, provider_url, pricing (jsonb), payment_rails (jsonb), payee, verification_level, health, last_verified_at, evidence (jsonb), enabled	Registry. Basis for the gate in C02 and the payee in C05.
jobs	id, mandate_id, goal, budget, status, created_at	Groups decisions and ledger entries for the job summary.
negotiations (P1)	id, job_id, round, offer (jsonb), ceiling, outcome	Round limit and price ceiling come from the mandate. One row per offer.
revenue_entries (P1)	id, job_id, receipt_hash, amount, currency, payer_ref, verified_at	Unique on receipt_hash. Credited only after receipt verification. Never changes a mandate budget.

12.2 HTTP API (v1)

Method and path	Purpose	Auth
POST /v1/mandates	Create a mandate (new version if the id exists).	Operator token
GET /v1/mandates/:id	Read a mandate and its remaining budget.	Operator or agent token
POST /v1/mandates/:id/revoke	Revoke immediately.	Operator token
POST /v1/authorize	Submit a challenge and context. Returns decision, checks, decisionId.	Agent token
POST /v1/payments/:decisionId/sign	Gateway asks the signer to sign. Internal only, not exposed externally.	Service token
POST /v1/payments/:decisionId/submit	Report the tx hash, start post-flight.	Agent token
GET /v1/decisions	List and filter decisions.	Operator token
GET /v1/approvals, POST /v1/approvals/:id/{approve,reject}	Human approval queue.	Operator token
POST /v1/freeze, POST /v1/unfreeze	Kill switch.	Operator token
GET /v1/audit, GET /v1/audit/verify	Read the chain and verify it.	Operator token
POST /v1/services, GET /v1/services, POST /v1/services/:id/probe	Register a service, list the registry, run the free endpoint probe.	Operator token
POST /v1/mandates/:id/delegate	Create a child mandate carved out of the parent's remaining budget.	Agent token (parent) or operator token
GET /v1/jobs, GET /v1/jobs/:id	Per-job summary: budget, committed, settled, released, remaining, blocked attempts.	Operator token
POST /v1/jobs/:id/deliverables	Submit a worker's deliverable. Runs the acceptance check and releases the final payment if it passes.	Agent token
POST /v1/negotiations/:id/offers	Submit or answer an offer. Enforces ceiling, round limit, payee and asset. Returns accept, counter or reject.	Agent token
POST /v1/jobs/:id/revenue	Report a receipt from the earn endpoint. Verifies it and credits a revenue entry, or rejects it.	Agent token
GET /healthz, GET /metrics	Liveness and metrics.	None / internal

Example: authorize

```
POST /v1/authorize
{ "mandateId": "mnd_01HZX...", "host": "api.example.com", "purpose": "buy 40 API calls",
  "challenge": { "id": "ch_8d1...", "method": "nearintents", "recipient": "0xDEP0...",
                 "amount": "0.250000", "currency": "USDC", "originNetwork": "eip155:42161",
                 "refundTo": "0xPAYER...", "expires": "2026-11-06T10:05:00Z" } }

200 { "decisionId": "dec_01HZY...", "decision": "ALLOW", "risk": 12,
      "checks": [ {"id":"C01","result":"pass"}, ... ], "bind": { ... } }
```

Errors use a single shape: `{ error: { code, message, check? } }`. A DENY is a normal 200 response with `decision: DENY`, not an HTTP error, so agents can handle it uniformly.

13. Security design

Concern	Design
Key custody	One signing key in the signer process. Loaded from an env file or secret mount with 0600 permissions. API and dashboard have no access. Float capped at about $20 on mainnet for the demo.
Decision binding	Signer reads the decision by id from the store, checks status ALLOW, not expired, nonce unused, and that every field of the transaction equals `bind`. Marks the nonce used in the same transaction as signing.
Process isolation	Signer listens on a Unix socket or localhost-only port, accepts only the gateway's service token, runs as a separate OS user in Docker Compose.
Authentication	Operator token for humans (dashboard), agent token per agent id for SDK, service token for gateway to signer. Tokens hashed at rest. Rate limited.
Input handling	Zod validation on every input. Size limits. Challenge parsing is strict and rejects unknown critical fields.
SSRF and fetch safety	Gateway only calls a fixed allowlist of hosts (1Click, configured RPC). The merchant URL is never fetched by the gateway.
Prompt-injection posture	The LLM never decides money movement. The decision comes from code over structured data. Free-text purposes are logged but never trusted.
Audit integrity	SHA-256 chain over canonical JSON. Verify command recomputes the chain. Optional on-chain head anchor (P2).
Secrets hygiene	.env in .gitignore, secret scanning in CI, fixtures scrubbed of keys and tokens. Demo mnemonic is never reused.
Supply chain	Pinned lockfile, minimal dependencies in the signer, `npm audit` in CI.
Human gates	Any mainnet transaction, deployment, public post or making the repo public is a manual step in the checklist, per hackathon agent rules.
Registry gate	A service must be enabled and verified before it can receive a payment. A change of payee or terms disables it until a human re-enables it.
Listing sanitization	Third-party listing text is stored for humans and never placed in a model prompt. The planner gets structured fields only. Covered by attack A16.
Delegation	Child mandates are carved out of the parent's remaining budget, with scope and expiry subsets, depth one and a cascading revoke. Covered by property tests.
Function-call keys	Not relied on for limits, because they cannot attach deposits or move tokens. Limits are enforced by the gateway and signer. A Day-0 check looks for a smaller role, for example limiting which methods a key may call.
Attestation	The demo agent's LLM runs on NEAR AI Cloud (TEE inference). Show its attestation in the dashboard if the Day-0 check confirms it can be fetched. A TEE for the signer is P2.

14. Red-team benchmark design

The benchmark is the product's proof. It must be fair, reproducible and honest.

14.1 Targets

Target	Description
T0 Naive agent	Follows any 402 challenge and pays. This is the 'before' picture.
T1 Stock mppx policy	Agent with the library's built-in policy (allowed networks and currencies, max amount, expected destination) configured as sensibly as possible.
T2 Tollgate	Agent protected by the full pipeline with a representative mandate.

14.2 Method

- A mock merchant returns a legitimate or malicious challenge per scenario. 1Click, quote and chain responses come from recorded fixtures.
- Each scenario has an expected outcome per target. The runner compares actual to expected and records the decision and the failed check.
- Outcome is 'paid', 'blocked' or 'held for approval'. Report the exact count for each target. If a baseline blocks an attack, say so.
- Benign controls B01-B05 measure false denies. A product that blocks everything is useless.
- Reports: console table, `bench/report.json`, `bench/report.html` (also shown in the dashboard).
- A mock directory serves poisoned listings for A16. All three targets are run against it and the result is reported whatever it is.

- **Live smoke test**: one real small payment, one real blocked attack, tx hashes pasted in the README. Not part of CI.

14.3 Fairness rules

- Baselines get the best reasonable configuration, documented in the repo.
- Scenarios are fixed before looking at results. No tuning scenarios to flatter Tollgate.
- Known limits are listed next to the results (section 8.3).

15. Dashboard UX

Screen	Content	Primary action
Overview	Spend vs budget, decisions today by outcome, open approvals, audit status badge, freeze switch.	Freeze, jump to approvals
Mandates	List and editor with JSON preview and plain-language summary ('Pay up to $5 at 2 hosts until 12 Nov').	Create, revoke
Decisions	Live feed. Each row expands to the 12 checks with evidence, risk and bind data.	Filter, open evidence
Approvals	Pending items with the reason they were held, amounts, payee, quote comparison.	Approve or reject with a reason
Audit	Chain list, verify button, first-broken-entry highlight, export.	Verify
Jobs	Per-job budget, committed, settled, released and remaining, revenue and net, the blocked attempts and the check that stopped each, the negotiation rounds and the delegation tree of hired workers.	Open a job
Registry	Services with verification level, health, quoted price, payee and last probe. Third-party text is shown to humans only.	Probe, enable, disable
Benchmark	Table of attacks by target with outcomes, bar of blocked counts, link to the JSON report.	Re-run offline

Design principles

- **Explain every decision.** The check that failed and the evidence are one click away.
- **Plain language first, JSON second.** Humans approve what they can read.
- **No color-only status.** Use labels and icons with color. Keyboard navigable.
- **Seed data included** so a first-time visitor sees a populated dashboard without funds.

16. Delivery plan

> **Week 0 (now to 31 Oct) is preparation, not building.** Do only what the rules plainly allow: accounts, API keys, reading docs, and disposable spike scripts you do not commit. The registry, the demo agent and every other piece of product code, and the first commit, happen on 1 Nov.

16.1 Week 0 preparation (26-31 Oct)

When	Task	Done when
Mon 26 Oct	Read this PRD. Ask organizers on Discord whether prep work is acceptable. Apply for the 1Click API key. Pick a distinctive project name (it must be unique across the hackathon).	Reply saved. Key application submitted. Name chosen with one backup.
Tue 27 - Wed 28	Throwaway spikes (not committed): 1Click status for PENDING_DEPOSIT, dry quote shape, minimum swap amount, whether wet quotes need a key, `sendDeposit` signature in mppx, `refundTo` semantics, NEP-141 storage deposit, whether the mppx server middleware can verify one small charge with a method we can also pay with (needed for earning). Free 402 probe of the candidate registry services: which payment methods each advertises and whether any offers NEAR Intents. Check whether NEAR AI Cloud attestation can be fetched. Decide MPP or x402.	One-page spike notes with answers. Plan adjusted if an assumption failed. Payment-path decision written down.
Wed 28 - Thu 29	Fund a mainnet wallet with about $25 of USDC (Arbitrum) plus gas, and a small separate customer wallet with about $3 for the earning demo. Create a separate receiving address for the agent. Create the mainnet NEAR account (it is also the prize payout address). Create a NEAR AI Cloud key. Set up an X post plan.	Wallet funded. NEAR account exists. Keys stored in a password manager.
Fri 30 Oct	Dry-run the toolchain: Node 22, pnpm, Docker, Postgres. Sketch the demo storyboard. Prepare a logo and a README skeleton in a notes file.	Toolchain works on a clean shell.
Sat 31 Oct	Rest. Re-read the live HACKATHON.md for changes. Confirm submission page access.	Ready to start at 00:00 UTC on 1 Nov (01:00 in Lagos).

16.2 Build window, day by day (1-14 Nov UTC)

Day	Work	Exit check
Sun 1 Nov	Repo, monorepo, CI, schema, mandate types, ledger skeleton with the proposed, committed and settled states.	Compose up. CI green.
Mon 2 Nov	Verifier package. C01-C04 with tests. Fixture record and replay.	Checks pass unit tests.
Tue 3 Nov	C05-C08 against fixtures. Mock merchants, including the negotiating seller agent (offer, counter, accept, walk, and its bad-faith strategies). Registry table with seeded entries and the registry gate in C02.	Pipeline blocks A01-A06 offline. An unverified service is refused. Seller scripts run on their own.
Wed 4 Nov	C09, C10, reserve-then-settle ledger, race test. Authorize endpoint.	Concurrent budget test passes.
Thu 5 Nov	Audit chain and verify CLI. Signer process and decision binding.	Tamper test fails verify. Signer refuses a modified tx.
Fri 6 Nov	SDK 402 wrapper. First live mainnet payment with a tiny amount through the full path. Build the earn endpoint (mppx server middleware) and, if no registry service is payable, use it as the merchant for this first live payment.	One real payment end to end. Tx hash saved. The earn endpoint issues a challenge and verifies one payment.
Sat 7 Nov	Benchmark runner: T0, T1, T2, A01-A16 (including the poisoned-listing scenario), B01-B05. First honest results.	Results table printed.
Sun 8 Nov	Dashboard shell, decisions feed, check breakdown, registry screen, jobs screen with the money states. **Cut line 1**: if behind, drop P1 items except approvals, negotiation, hiring and earning.	Feed shows seeded and live data.
Mon 9 Nov	Approvals (C12), mandate editor with price ceiling and round limit. Negotiation in the gateway: offers table, ceiling and round-limit enforcement, offer-to-challenge match. Scenario tests for each seller strategy.	Over-ceiling counter, payee swap and late price raise are all denied. Approve flow works.
Tue 10 Nov	Demo builder agent on NEAR AI Cloud: decides and buys through registry services, **negotiates** with the seller, prompt-injection scene, blocked over-budget purchase. Post-flight P01-P04.	`pnpm demo` runs start to finish and closes a negotiated deal.
Wed 11 Nov	Delegated mandates, two or three hired workers, staged payments, acceptance check, invariant tests. **Cut line 2** at day's end: if hiring is not working, simplify to one worker and add no more features.	Property tests show the parent budget is never exceeded. Demo hires and pays in stages.
Thu 12 Nov	**Earning** end to end: the agent sells its report through the earn endpoint, the customer agent pays, the gateway verifies the receipt and credits revenue, the job summary shows spend, revenue and net. Then hardening: fault injection, fail-closed tests, secret scan, README for humans and machines, fresh-machine test. Kill switch and metered compute only if everything above is green.	Revenue equals the verified receipt. CI green. Clean clone to results under 5 minutes.
Fri 13 Nov	Record the 2-minute demo, X post draft, live smoke test with hashes, claims checklist, final polish. **Target submit.**	Submission sent with time to spare.
Sat 14 Nov	Buffer only. Fix a submission problem, never start a feature. Deadline 23:59 UTC (00:59 WAT on 15 Nov).	Nothing new merged.

Colors: green foundations, blue signing and proof, amber product surface, purple hiring and hardening, red delivery. The schedule assumes about 8-10 focused hours a day, and v1.2 leaves little slack on Days 8-12 beyond Saturday's buffer. If a day is lost, drop items in this order: earn-and-reinvest rule, metered compute (use the labelled mock), NEAR AI Cloud attestation display, post-flight anomaly flags, kill switch. Then simplify, never remove, the brief-critical items: hiring to one worker, negotiation to a single counter-offer round, earning to a labelled simulated customer. Core, signer, benchmark, the basic demo agent and the three brief-critical items are never dropped outright.

16.3 Milestones

Milestone	Date	Meaning
M1 Verifier proven	Wed 4 Nov	Blocks A01-A13 offline. Without this there is no project.
M2 Real payment	Fri 6 Nov	A live end-to-end payment with a signed, decision-bound transaction.
M3 Evidence	Sat 7 Nov	Benchmark table with honest numbers for three targets, 16 attacks.
M4 Product and negotiation	Tue 10 Nov	Dashboard, approvals and the demo agent working, including a negotiated deal with the ceiling enforced.
M5 Hiring	Wed 11 Nov	Delegated mandates and hired workers working, invariants tested.
M6 Earning	Thu 12 Nov	A report sold through the earn endpoint, receipt verified, revenue shown next to spend. The full economy loop runs in one command.
M7 Ship	Fri 13 Nov	Repo public (after confirmation), X post up, form submitted.

17. Testing, QA and test budget

Level	What	Tooling
Unit	Every check has pass and fail fixtures. Ledger and audit are pure and heavily tested.	Vitest
Property	Ledger never exceeds budget under random concurrent reserves. Child mandates never exceed the parent's remaining budget, and revoke cascades. Audit chain verify passes after any valid append sequence.	fast-check
Integration	Gateway with Postgres and mock merchants. Signer over its socket. An unverified registry service is never paid.	Vitest plus Docker Compose
Fault injection	1Click timeout, malformed JSON, 5xx, slow responses. Expected result is always DENY or NEEDS_APPROVAL, never ALLOW.	Mock adapters
End to end (offline)	Full bench with fixtures.	`pnpm bench`
Live smoke	One real payment and one blocked attack with small amounts. Manual checklist with human confirmation of each spend.	Manual
Security	Secret scan, dependency audit, signer negative tests (tampered amount, payee, chain, reused nonce).	gitleaks, npm audit, Vitest
Accessibility	Dashboard keyboard pass and automated audit.	Lighthouse

Mainnet test budget (estimate)

Item	Estimate
USDC for live payments and a few failed or refunded attempts	$10-15
Gas on Arbitrum	$2-3
NEAR account storage and NEP-141 storage deposits	$1-2
Reserve for retries and unexpected minimums	$5-8
Customer wallet for the earning demo (a few small purchases of the report)	$2-3
Total	**About $22-33**

Numbers are estimates until the Week-0 spikes confirm minimum swap amounts. Use the smallest amounts the API accepts. Never run live tests from automated agents without a human confirming each transaction. Purchases from real registry vendors and any paid compute job are not included. Prices were not visible in the directory, so cap them from the reserve line.

18. Deployment and DevOps

This is where your backend and DevOps experience shows. Treat the repo as a small production system.

Area	Plan
Repo layout	pnpm monorepo: `packages/core` (checks, ledger, audit), `packages/sdk`, `apps/gateway`, `apps/signer`, `apps/dashboard`, `packages/registry`, `apps/demo-agent`, `apps/workers`, `apps/mock-merchants`, `bench/`, `docs/`.
Local run	`docker compose up` starts Postgres, gateway, signer, dashboard and the mock merchants. `pnpm demo` runs the demo agent. Offline mode needs no keys.
CI (GitHub Actions)	Lint, typecheck, unit and property tests, coverage gate, offline bench, secret scan, dependency audit, Docker build.
Config	Twelve-factor env vars, `.env.example` committed, real `.env` ignored. Zod-validated config at boot with clear errors.
Hosting (optional)	A small VPS or container host for a public demo: gateway and dashboard behind a reverse proxy with TLS, signer not exposed. Deploy is a manual step per hackathon agent rules.
Database	Drizzle migrations. Seed script for demo data. Nightly dump is optional.
Logging	JSON logs with request and decision ids. No secrets, tokens or full keys in logs (tested).
Releases	Tag v1.0.0 on submission. Changelog. The submission commit hash goes in the X post and the form.

19. Risks and open questions

Risk	Likelihood	Impact	Mitigation
Prep before 1 Nov judged as breaking 'build during the hackathon'	Low-Med	High	Only docs, accounts and uncommitted spikes. Ask organizers now and keep the reply.
1Click API key approval is slow or denied	Med	Low	Works without a key at 0.2% fee. Document the difference.
Spike shows status endpoint lacks fields needed by C06	Med	High	Fallback: verify via deposit address match and own quote only. Adjust acceptance criteria.
mppx `sendDeposit` or EVM path behaves differently than documented	Med	Med	Day-0 spike. Fallback: own thin client for the payment step.
No testnet makes live testing costly or risky	Certain	Med	Offline fixtures, strict small amounts, float cap, manual confirmation.
Solo schedule slips	High	High	Two cut lines, P1 order of drop, buffer day, submit target Day 13.
Another team ships a similar firewall	Med	Med	Differentiate on benchmark evidence, wallet-agnostic design and polish.
Judges cannot run it	Low	High	One command, offline mode, fresh-machine test on Day 12.
Mainnet funds lost in a bug	Low	Low	Small float only. Kill switch. Decision-bound signer.
Submission cannot be edited	Certain	High	Dry-run the form fields on Day 12. Second person or reviewer proofreads.
Registry services do not accept NEAR Intents, or need an API key or account	Med	Med	Day-0 402 probe. Fall back to our own mock merchants for the NEAR Intents path. Label anything simulated.
Scope creep from hiring, compute, negotiation and earning	High	High	Tiers and drop order in section 16. Core, signer and benchmark are never traded for extras. Brief-critical items are simplified, not removed.
The paid-endpoint server cannot verify a payment method we can pay with	Med	Med	Week 0 spike on the mppx server side. Use whichever method works and say so. Fallback: a labelled simulated customer with a replayed receipt.
Days 8-12 are full after adding negotiation and earning	High	Med	Seller agent built early with the mock merchants; earn endpoint used as the live-payment merchant on Day 6; tests written with each feature; drop order protects the brief-critical items.
Judges see negotiation as scripted rather than real	Med	Low	Label the seller as our own agent. Show real model decisions and the code-enforced ceiling denying a bad counter.
Project name already taken by another team	Low-Med	Med	Pick a distinctive name with a backup. Submit by 13 Nov so a clash can be fixed before the deadline.
Effort split across two payment protocols (MPP and x402)	Med	Med	One payment path. Decide after the Day-0 probe and write the decision down.

Risk	Likelihood	Impact	Mitigation
Poisoned listing or directory text reaches the planner	Med	High	The planner sees structured fields only. Attack A16 tests it.

Open questions (answer in Week 0)

- Is preparation before 1 Nov acceptable under the rules? (Ask organizers.)
- Does the 1Click status response for PENDING_DEPOSIT include the original quote request? (Needed by C06.)
- Do wet (non-dry) quotes require an API key, and what is the minimum swap amount?
- Does `refundTo` semantics allow the payer to be verified from the challenge alone?
- Is the mppx `sendDeposit` callback signature as documented for the EVM path?
- Does the NEAR side need a NEP-141 storage deposit for the merchant account?
- Which candidate registry services offer a payment method Tollgate supports, and does any offer NEAR Intents? (Free 402 probe.)
- Can the mppx server middleware accept and verify one small charge with a method we can also pay with? (Needed for earning.)
- Can the NEAR AI Cloud attestation be fetched for display?
- Is the chosen project name free, and is there a backup?

20. Submission and demo plan

20.1 Submission checklist (target Fri 13 Nov)

The submission is one POST to the hackathon API (or the same fields on the form). It cannot be edited, and a second attempt with the same project name or email is rejected. Every value below is the human's to decide; nothing is guessed or filled in for you.

Field	Rule	Plan
projectName	2-80 characters, unique across the hackathon	A distinctive name chosen in Week 0, with a backup.
leadName, email	Name 2-80 characters. Email valid, unique, max 160	Yours.
size	One of 1, 2 - 4, 4 - 6, 6 - 8, 8+ (exact text)	1
tracks	A list of track ids	`security` and `agents`
nearAddress	Mainnet NEAR account for the prize payout	Created by you in Week 0. Never created for you.
projectUrl	Public repository URL	Repo is made public by you, after confirmation, by 14 Nov 23:59 UTC.
demoUrl	Public X post with video or screenshots, saying it was built at NEAR HACKATHON	You write and post it. A draft is prepared on Day 12.
idea	20-2,000 characters: the problem, the product, how NEAR fits	Outline in 20.4. You approve the final text.
consent	True, only after you agree to the organizers' contact notice	You read and confirm it.
requestId	UUID generated once and reused on retries	Generate once. Do not commit the payload (it holds personal data).

- Before sending, print the full payload, check it against the live HACKATHON.md and confirm it. Tag v1.0.0 and use the same commit hash in the X post and the form.
- Run a fresh-clone test on a clean machine on Day 12. Submit by Fri 13 Nov so a rejected field or a name clash can still be fixed before the deadline.

20.2 Two-minute demo storyboard

The story is one small job. The figures below are illustrative; the video uses the real numbers from the run.

Time	Scene	On screen
0:00-0:15	Hook: 'We gave an AI agent $8 and a job. A fake seller asked for $18.'	Naive agent pays the fake request. Replayed, no real funds.
0:15-0:30	Tollgate in one line and a 10-line mandate, including a price ceiling.	Mandate JSON and its plain-language summary.
0:30-0:50	The demo agent decides what to buy, negotiates with a seller (opens high, settles lower), then pays through a registry service.	Negotiation rounds, the agreed price, 12 green checks, tx hash.
0:50-1:05	It hires a worker with a smaller budget carved out of its own and pays in two stages.	Child mandate and delegation tree.
1:05-1:20	It sells its finished report from its own paid endpoint. A customer agent pays.	Receipt verified, revenue line appears next to the spend.
1:20-1:40	The $18 purchase and other attacks are blocked: seller raising the price past the ceiling, payee swap, refund hijack, poisoned listing.	Red DENY rows with the failed check and evidence. Audit entry.
1:40-1:55	Job summary and the benchmark table: naive vs stock policy vs Tollgate.	Spend, revenue, net, blocked. Real numbers from the run.
1:55-2:00	Wallet-agnostic, open source, an audit you can verify. Repo link.	Audit verify, logo, URL.

20.3 README for humans and AI judges

- Top: one-sentence pitch, results table, 15-second quick start.
- Numbered steps in the 'do X, see Y' style: `pnpm i`, `pnpm bench`, `pnpm demo`, expected output shown. Include a short table mapping each verb in the Agent Economy brief to the command that shows it.
- Name the NEAR tools used (NEAR Intents and the 1Click flow, NEAR AI Cloud, and why function-call keys were not enough), as the hackathon page asks.
- Architecture diagram, threat model link, limits and honest non-goals, how to run live (manual, small amounts). Label anything simulated.
- A machine-readable `AGENTS.md` stating what an automated agent may and may not do (no signing, spending, deploying or posting without a human).

20.4 Project description outline (the idea field)

- **Problem** (1-2 sentences): agents trust whatever payment request a server sends; spending caps do not check the request itself.
- **Product**: Tollgate, a payment firewall in front of any wallet, with a mandate, 12 checks, a decision-bound signer and a verifiable audit log.
- **How NEAR fits**: NEAR Intents and the 1Click API for settlement checks and a real mainnet payment, NEAR AI Cloud for the demo agent, and the finding that function-call keys cannot enforce spend limits.
- **The agent economy loop**: a demo agent that decides, signs through the guarded signer, negotiates, hires and earns, so the brief's verbs are shown, not claimed.
- **Evidence**: the 16-attack benchmark against two baselines, one real payment and one blocked attack with tx hashes, and a job summary with spend, revenue and net.

21. Portfolio packaging

The hackathon is the deadline. The portfolio is the lasting value. Package it for a reader who has two minutes.

- **Case study page** (800 words): the problem, the decision to pivot after finding OutLayer, the design, the benchmark numbers and what failed.
- **Pinned repo** with a short, honest README, diagrams from this document and a visible CI badge.
- **Short video** on the repo and the X post. Add a longer technical walkthrough later if time allows.
- **Write-up** on how verification differs from limits. Your mass communication training is a real edge here: clear writing and a good demo are what make technical work legible to judges and employers.
- **Skills shown**: backend design, concurrency (ledger), security thinking, testing discipline, DevOps (Compose, CI, config), API design and applied agent security.

After the hackathon

- Publish the SDK as an npm package after a security review.
- Add the OutLayer adapter and a NEAR-origin signer.
- Propose the challenge verification checks upstream as guidance for MPP client implementers.

Appendix A. Sources

Read during research for this document. Re-check details on Day 0; hackathon rules and APIs can change.

Source	Location
NEAR HACKATHON site and rules	https://nearhackathon.com
HACKATHON.md (rules, submission, agent guidance)	https://nearhackathon.com/HACKATHON.md
NEAR Intents and 1Click API	https://docs.near-intents.org
1Click API endpoint	https://1click.chaindefuser.com
NEAR Intents partner portal (API key)	https://partners.near-intents.org
Machine Payments Protocol (MPP)	https://mpp.dev
NEAR Intents MPP SDK	@defuse-protocol/nearintents-mpp-sdk (npm)
OutLayer	https://outlayer.ai
NEAR AI Cloud	https://cloud-api.near.ai/v1
NEAR docs (access keys, NEP-413)	https://docs.near.org
MPP services directory	https://mpp.dev/services
MPP micropayments and sessions	https://mpp.dev/use-cases/micropayments
x402 directories (from attached notes, not verified)	x402.new, x402-list.com, payapi.market, the402.ai, x402.pub

Appendix B. Glossary

Term	Meaning
Agent	Software driven by an LLM that takes actions, including payments.
MPP	Machine Payments Protocol. A merchant issues a payment challenge, the client pays and presents a credential.
Challenge	The merchant's payment request: who to pay, how much, in what asset, until when.
1Click	NEAR Intents' swap API that quotes, assigns a single-use deposit address and reports status.
Deposit address	A single-use address that receives the payer's funds for one quote.
Mandate	The human's machine-readable spending authority for an agent.
Decision-bound signing	The signer signs only what a stored ALLOW decision covers, once.
Function-call key	A restricted NEAR access key. It cannot attach deposits or transfer tokens.
Fail closed	On error or ambiguity, refuse rather than allow.
TOFU	Trust on first use. Pin a value the first time it is approved.
TEE	Trusted execution environment. Hardware-isolated computation with attestation.
Service registry	Tollgate's curated table of services the agent may pay, each with a verification level.
Child mandate	A smaller mandate carved out of a parent's remaining budget and given to a hired worker.
Money states	Proposed (awaiting a decision), committed (reserved), settled (confirmed and spent) and released (returned).
x402	Another HTTP 402 payment protocol, used by many pay-per-call directories. Different from MPP.
Price ceiling	The highest price a negotiation may reach, set in the mandate and enforced in code.
Round limit	The maximum number of offers and counter-offers in one negotiation, set in the mandate.
Revenue entry	A ledger record of money the agent earned, created only after the gateway verifies a fresh, settled receipt.
Earn endpoint	The agent's own paid HTTP endpoint that sells its report using an MPP 402 challenge.